

GUÍA COMPLETA

CRUD con Spring Boot + Thymeleaf + JPA (MySQL)

Gestión de Servicios de TICS — Laboratorios

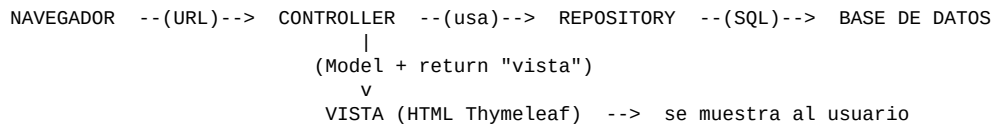
Todo lo que te pueden pedir crear, con el código y DÓNDE cambiarlo.

CONTENIDO

1. Cómo funciona (las 3 capas)
2. Configuración inicial (pom, properties)
3. Crear la BASE DE DATOS y la TABLA
4. La ENTITY (mapear la tabla)
5. El REPOSITORY
6. El CONTROLLER (listar, registrar, editar, borrar)
7. Las VISTAS Thymeleaf (tabla, formulario, editar)
8. Cómo LINKEAR todo (rutas y botones)
9. BUSCAR / filtrar (Query Methods)
10. REPORTE (máx, mín, promedio, total)
11. Flujo de GIT (main / develop / feature)
12. ERRORES comunes (los que quitan puntos)
13. CHULETA rápida

1. Cómo funciona: las 3 capas

Todo CRUD en Spring Boot MVC se arma con **3 piezas** que se conectan en cadena. Si entiendes este flujo, resuelves cualquier problema:



Las piezas:

- **ENTITY** — una clase Java que representa una tabla (cada objeto = una fila).
- **REPOSITORY** — interfaz que da acceso a la BD (findAll, save, findById, deleteById...).
- **CONTROLLER** — recibe la URL, usa el repository y devuelve una vista.
- **VISTA** — el HTML con Thymeleaf que ve el usuario.

Nota: Orden para crear siempre: **1) Entity → 2) Repository → 3) Controller → 4) Vista.**

2. Configuración inicial

2.1 pom.xml — las 4 dependencias que SIEMPRE necesitas

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId> <!-- Web / controladores -->
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId> <!-- Vistas HTML -->
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId> <!-- Repositorios / BD -->
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId> <!-- Driver MySQL -->
  <scope>runtime</scope>
</dependency>
```

Nota: Si faltan, sale error. Después de agregarlas: **Reload Maven** (ícono m, Reload All Maven Projects).

2.2 application.properties

Va en src/main/resources/application.properties. **CAMBIA** el nombre de la BD, usuario y password:

```
spring.application.name=lab

# La URL va en UNA sola línea (aquí se muestra cortada por espacio):
spring.datasource.url=jdbc:mysql://localhost:3306/midb?serverTimezone=America/Lima&useSSL=false&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=TU_PASSWORD

# Hibernate crea/actualiza las tablas solas desde tus @Entity:
spring.jpa.hibernate.ddl-auto=update
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Qué cambiar: midb (nombre BD), root (usuario), TU_PASSWORD.

3. Crear la BASE DE DATOS y la TABLA

En MySQL Workbench. La base de datos SIEMPRE la creas tú; la tabla la puedes crear tú o dejar que Hibernate la cree.

3.1 Crear la base de datos (obligatorio)

```
CREATE DATABASE midb;
USE midb;
```

3.2 Crear la tabla — opción A: a mano con SQL

```
CREATE TABLE mascotas (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  especie     VARCHAR(50),
  edad        INT,
  nombre_dueno VARCHAR(100),
  estado      VARCHAR(30)
);

-- Datos de prueba (opcional):
INSERT INTO mascotas (nombre, especie, edad, nombre_dueno, estado) VALUES
('Firulais', 'Perro', 3, 'Luis', 'Activo'),
('Michi', 'Gato', 5, 'Ana', 'Inactivo');
```

3.3 Crear la tabla — opción B: automática (Hibernate)

Si en el properties pones `spring.jpa.hibernate.ddl-auto=update`, Hibernate **crea la tabla sola** a partir de tu `@Entity`. Solo necesitas la base de datos vacía.

Nota: Tipos: **VARCHAR**→String · **INT**→Integer · **DECIMAL**→Double/BigDecimal · **DATE**→LocalDate. La PK con `AUTO_INCREMENT = @GeneratedValue(IDENTITY)`.

4. La ENTITY (mapear la tabla)

Va en el paquete entity. Cada atributo = una columna.

```
package com.example.lab.entity;

import jakarta.persistence.*;           // OJO: jakarta (Boot 3/4), NO javax

@Entity
@Table(name = "mascotas")               // <-- CAMBIA: nombre de la TABLA
public class Mascota {                  // <-- CAMBIA: nombre de la CLASE

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  @Column(name = "id")
  private Integer id;

  @Column(name = "nombre")              // <-- name = columna EXACTA en la BD
  private String nombre;

  @Column(name = "edad")
  private Integer edad;                 // usa Integer (no int) por los NULL

  @Column(name = "nombre_dueno")        // BD con guion_bajo, atributo camelCase
  private String nombreDueno;

  // ... mas campos ...

  // GETTERS Y SETTERS: genera con Alt+Insert (!SIEMPRE!)
  public Integer getId() { return id; }
  public void setId(Integer id) { this.id = id; }
  public String getNombre() { return nombre; }
  public void setNombre(String nombre) { this.nombre = nombre; }
  public Integer getEdad() { return edad; }
  public void setEdad(Integer edad) { this.edad = edad; }
  public String getNombreDueno() { return nombreDueno; }
  public void setNombreDueno(String n) { this.nombreDueno = n; }
```

```
}
```

Qué cambiar: nombre de la clase, `@Table(name)`, y cada campo con su `@Column(name)`.

Nota: Regla de oro: el name de `@Column` debe ser **idéntico** a la columna de la BD. Y SIEMPRE getters/setters, o Thymeleaf falla con EL1008E.

5. EL REPOSITORY

Va en el paquete `repository`. Es una **interfaz** de una línea. Con eso ya tienes `findAll`, `save`, `findById`, `deleteById`, `count`... gratis.

```
package com.example.lab.repository;

import com.example.lab.entity.Mascota;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface MascotaRepository extends JpaRepository<Mascota, Integer> {
    //                ^Entidad    ^tipo del @Id
}
```

Qué cambiar: el nombre del repo, la entidad (`Mascota`) y el tipo del `@Id` (`Integer`).

6. EL CONTROLLER (listar, registrar, editar, borrar)

Va en el paquete `controller`. Aquí está TODO el CRUD. Cambia la ruta base `/mascota` por la tuya.

```
package com.example.lab.controller;

import com.example.lab.entity.Mascota;
import com.example.lab.repository.MascotaRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Optional;

@Controller
@RequestMapping("/mascota") // <-- CAMBIA: ruta base
public class MascotaController {

    @Autowired
    MascotaRepository mascotaRepository; // <-- CAMBIA: tu repositorio

    // ---- LISTAR ----
    @GetMapping("/listar")
    public String listar(Model model) {
        List<Mascota> lista = mascotaRepository.findAll();
        model.addAttribute("lista", lista); // <-- nombre para la vista
        return "mascota/lista"; // <-- vista: templates/mascota/lista.html
    }

    // ---- MOSTRAR FORMULARIO (registrar) ----
    @GetMapping("/nuevo")
    public String nuevo() {
        return "mascota/new";
    }

    // ---- GUARDAR (registrar o actualizar) ----
    @PostMapping("/guardar")
    public String guardar(Mascota mascota) { // Spring rellena el objeto (data binding)
        mascotaRepository.save(mascota); // sin id = INSERT ; con id = UPDATE
        return "redirect:/mascota/listar"; // redirige al listado
    }

    // ---- EDITAR (carga los datos actuales) ----
    @GetMapping("/editar")
```

```

    public String editar(@RequestParam("id") Integer id, Model model) {
        Optional<Mascota> opt = mascotaRepository.findById(id);
        if (opt.isPresent()) {
            model.addAttribute("mascota", opt.get());
            return "mascota/edit";
        }
        return "redirect:/mascota/listar";
    }

    // ---- BORRAR ----
    @GetMapping("/borrar")
    public String borrar(@RequestParam("id") Integer id) {
        mascotaRepository.deleteById(id);
        return "redirect:/mascota/listar";
    }
}

```

Qué hace cada método:

- **listar**: trae todo con `findAll()` y lo manda a la vista.
- **nuevo**: muestra el formulario vacío.
- **guardar**: recibe el form (POST), `save()` inserta o actualiza, y redirige.
- **editar**: busca por id (`findById`→`Optional`) y manda los datos a la vista.
- **borrar**: elimina por id y redirige.

Nota: `save()` es inteligente: si el objeto NO tiene id → INSERT (registrar); si tiene id → UPDATE (editar). Por eso el mismo método sirve para registrar y editar.

7. Las VISTAS Thymeleaf

Van en `src/main/resources/templates/mascota/`. El nombre del archivo debe coincidir con el return del controller.

7.1 lista.html — LISTAR en tabla

```

<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"><title>Lista</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container">
    <h2>Lista de Mascotas</h2>
    <a class="btn btn-success mb-3" th:href="@{/mascota/nuevo}">+ Nueva</a>

    <table class="table table-striped">
        <thead>
            <tr><th>ID</th><th>Nombre</th><th>Edad</th><th></th><th></th></tr>
        </thead>
        <tbody>
            <tr th:each="m : ${lista}">                                <!-- recorre la lista -->
                <td th:text="${m.id}"></td>
                <td th:text="${m.nombre}"></td>
                <td th:text="${m.edad}"></td>
                <td><a th:href="@{/mascota/editar?id=' + ${m.id}'}" class="btn btn-primary btn-sm">Editar</a></td>
                <td><a th:href="@{/mascota/borrar?id=' + ${m.id}'}" class="btn btn-danger btn-sm">Eliminar</a></td>
            </tr>
        </tbody>
    </table>
</div>
</body>
</html>

```

Clave: `th:each="m : ${lista}"` repite la fila. `${m.nombre}` usa el atributo Java (camelCase).

7.2 new.html — REGISTRAR (formulario)

```
<form method="post" th:action="@{/mascota/guardar}">
  <label>Nombre</label>
  <input type="text" name="nombre" class="form-control">      <!-- name = atributo Java -->

  <label>Edad</label>
  <input type="number" name="edad" class="form-control">

  <label>Nombre del dueño</label>
  <input type="text" name="nombreDueno" class="form-control"> <!-- camelCase EXACTO -->

  <label>Estado</label>
  <select name="estado" class="form-select">
    <option value="Activo">Activo</option>
    <option value="Inactivo">Inactivo</option>
  </select>

  <button type="submit" class="btn btn-primary">Registrar</button>
</form>
```

Nota: Data binding: cada name del input debe ser IGUAL al atributo Java de la entidad (nombreDueno, no nombre_dueno). Si te equivocas, ese campo llega null.

7.3 edit.html — EDITAR (form con datos cargados)

```
<form method="post" th:action="@{/mascota/guardar}" th:object="${mascota}">
  <input type="hidden" th:field="*{id}">      <!-- id oculto = UPDATE -->

  <label>Nombre</label>
  <input type="text" th:field="*{nombre}" class="form-control">

  <label>Edad</label>
  <input type="number" th:field="*{edad}" class="form-control">

  <button type="submit" class="btn btn-primary">Actualizar</button>
</form>
```

Diferencia con new: se usa `th:object="${mascota}"` y `th:field="*{campo}"` para **precargar** los datos. El input hidden del id hace que `save()` haga UPDATE.

8. Cómo LINKEAR todo

Todo se conecta por las **rutas (URLs)**. Así encaja cada pieza:

Acción	Botón/link en la vista	Ruta en el controller
Ir al listado	@{/mascota/listar}	@GetMapping("/listar")
Nuevo (form)	@{/mascota/nuevo}	@GetMapping("/nuevo")
Guardar (submit)	th:action=@{/mascota/guardar}	@PostMapping("/guardar")
Editar	@{'/mascota/editar?id='+\${m.id}}	@GetMapping("/editar") + @RequestParam
Borrar	@{'/mascota/borrar?id='+\${m.id}}	@GetMapping("/borrar") + @RequestParam

Regla clave: el return "carpeta/archivo" del controller debe coincidir con la ruta del HTML en templates/. Ej: return "mascota/lista" → templates/mascota/lista.html.

Nota: redirect vs vista: return "mascota/lista" muestra un HTML. return "redirect:/mascota/listar" reenvía a OTRA ruta del controller (se usa después de guardar/borrar).

9. BUSCAR / filtrar (Query Methods)

Si te piden una **búsqueda**, declaras métodos en el repository y Spring genera el SQL. Formato: tipoRetorno findByAtributo(tipo param).

```
public interface MascotaRepository extends JpaRepository<Mascota, Integer> {

    // WHERE nombre LIKE %texto% (sin importar mayus/minus)
    List<Mascota> findByNombreContainingIgnoreCase(String nombre);

    // WHERE especie = ?
    List<Mascota> findByEspecie(String especie);

    // WHERE edad > ?
    List<Mascota> findByEdadGreaterThan(Integer edad);

    // ORDER BY edad DESC
    List<Mascota> findByOrderByEdadDesc();
}
```

En el **controller**, un método de búsqueda:

```
@GetMapping("/buscar")
public String buscar(@RequestParam("q") String q, Model model) {
    List<Mascota> lista = mascotaRepository.findByNombreContainingIgnoreCase(q);
    model.addAttribute("lista", lista);
    return "mascota/lista";    // reutiliza la misma tabla
}
```

Y en la vista, un formulario de búsqueda:

```
<form method="get" th:action="@{/mascota/buscar}">
    <input type="text" name="q" placeholder="Buscar...">
    <button type="submit">Buscar</button>
    <a th:href="@{/mascota/listar}">Limpiar</a>    <!-- vuelve a mostrar todo -->
</form>
```

Keywords útiles: Containing, IgnoreCase, GreaterThan, LessThan, Between, OrderByXDesc, countBy, existsBy.

10. REPORTE (máx, mín, promedio, total)

Con @Query (JPQL) haces consultas de agregación:

```
public interface MascotaRepository extends JpaRepository<Mascota, Integer> {
    @Query("SELECT MAX(m.edad) FROM Mascota m") Integer edadMaxima();
    @Query("SELECT MIN(m.edad) FROM Mascota m") Integer edadMinima();
    @Query("SELECT AVG(m.edad) FROM Mascota m") Double edadPromedio();
    @Query("SELECT COUNT(m) FROM Mascota m") Long cantidadTotal();
}
```

Nota: En JPQL usas la **entidad** (Mascota) y el **atributo** (m.edad), NO la tabla/columna. Importa org.springframework.data.jpa.repository.Query.

Controller:

```
@GetMapping("/reporte")
public String reporte(Model model) {
    model.addAttribute("max", mascotaRepository.edadMaxima());
    model.addAttribute("min", mascotaRepository.edadMinima());
    model.addAttribute("prom", mascotaRepository.edadPromedio());
    model.addAttribute("total", mascotaRepository.cantidadTotal());
    return "mascota/reporte";
}
```

Vista (promedio con 2 decimales):

```
<p>Edad maxima: <span th:text="${max}"></span></p>
<p>Edad minima: <span th:text="${min}"></span></p>
<p>Promedio: <span th:text="${#numbers.formatDecimal(prom, 1, 2, 'POINT')}"></span></p>
<p>Total: <span th:text="${total}"></span></p>
```

El #numbers.formatDecimal(valor, 1, 2, 'POINT') muestra 2 decimales con punto (ej. 5.20).

11. Flujo de GIT (main / develop / feature)

La gestión de ramas suele ser **parte de la nota**. Estructura: main (estable) → develop (integración) → feature/* (cada funcionalidad).

11.1 Preparar el repo (una sola vez)

```
git init
git add .
git commit -m "Proyecto base"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/TU_REPO.git
git push -u origin main
git checkout -b develop
git push -u origin develop
```

11.2 Por CADA funcionalidad

```
# 1) crear la rama feature desde develop
git checkout develop
git checkout -b feature/listado-mascotas

# 2) ... programar la funcionalidad ... luego guardar:
git add .
git commit -m "P2: listado de mascotas"

# 3) integrar a develop
git checkout develop
git merge feature/listado-mascotas
git push origin develop
```

Nota: Nombres de ramas típicos: feature/listado-mascotas, feature/registro-mascota, feature/edicion-mascota, feature/reporte-mascotas.

12. ERRORES comunes (los que quitan puntos)

- **javax vs jakarta** — Boot 3/4 usa `jakarta.persistence.*`, NO `javax`. Alt+Enter y elige `jakarta`.
- **@ID / @Column** — Es `@Id` (no `@ID`) y `@Column` (no `@Colum`). Un carácter rompe todo.
- **Falta dependencia** — Sin `spring-boot-starter-data-jpa` o `mysql-connector-j` en el pom → error al arrancar.
- **url not specified** — El `application.properties` está vacío o sin la URL de la BD.
- **@Column ≠ columna** — El name debe ser idéntico a la BD (ojo la ñ: `nombre_dueno`).
- **EL1008E** — Falta el **getter** de un campo. Genera getters/setters con Alt+Insert.
- **Template not found** — La vista no está en `templates/` o el nombre no coincide con el return.
- **Campo llega null** — El name del input no coincide con el atributo Java (data binding).
- **Port 8080 in use** — Ya hay una instancia corriendo. Detén la anterior (botón rojo Stop).

13. CHULETA rápida

Orden para resolver cualquier CRUD:

- 1) Crear BD (CREATE DATABASE) y configurar `application.properties`.
- 2) **Entity** (`@Entity`, `@Table`, `@Id`, `@Column`) + getters/setters.
- 3) **Repository** (extends `JpaRepository`).
- 4) **Controller** (listar → nuevo/guardar → editar → borrar).

5) **Vistas** en templates/carpeta/ (lista, new, edit).

6) Correr  y probar en el navegador.

7) Commit + merge a develop.

Métodos del repository (gratis): findAll() · save() · findById() · deleteById() · count() · existsById()

Anotaciones del controller: @Controller · @RequestMapping · @GetMapping · @PostMapping · @RequestParam · @Autowired

Thymeleaf: th:each · th:text · th:href · th:action · th:object · th:field · @{ruta} · \${var}